

LEVEL 11

Chapter 12: Cloud System Design (AWS)

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 12: Cloud System Design (AWS)

← [Chapter 11: Observability](#) · [Index](#) · Next → [Chapter 13: Architecture Patterns](#)

You already run EKS/ECS/Terraform in production, so treat this chapter as a reference map connecting every generic HLD concept from Chapters 4–11 to its concrete AWS implementation — exactly the translation interviewers want to hear when they ask "how would you actually build this."

12.1 Compute

Service	Problem it solves	When to use	When NOT to use	Alternatives
EC2	Raw virtual machines, full control	Need OS-level control, custom software, licensing constraints, or workloads that don't fit containers/serverless well	You want to avoid managing OS patching/scaling yourself	ECS/EKS (containers), Lambda (serverless)
ECS	Managed container orchestration, AWS-native	Simpler container orchestration needs, tighter AWS integration (IAM roles per task, Fargate for serverless containers), smaller platform teams	You need portability across clouds or the full Kubernetes ecosystem (Helm charts, operators, CRDs)	EKS, Fargate
EKS	Managed Kubernetes	Multi-cloud portability, complex orchestration needs, existing Kubernetes expertise/tooling (your own background), large-scale microservices	Small teams without Kubernetes operational maturity — the control-plane learning curve isn't free even though AWS manages the control plane itself	ECS (simpler), Fargate (serverless, less to manage)
Lambda	Serverless functions, event-driven, pay-per-invocation	Sporadic/bursty workloads, event-driven glue (S3 upload triggers processing, API Gateway backing a simple endpoint), unpredictable/spiky traffic where idle capacity cost matters	Long-running processes (15-min hard execution limit), workloads needing consistent low-latency warm starts at high sustained throughput (cold starts add latency), or heavy stateful compute	ECS/EKS for sustained, predictable workloads

Cost/scaling framing for interviews: EC2/EKS/ECS costs scale roughly with provisioned capacity (you pay for what you reserve, whether used or not, unless using auto-scaling well); Lambda costs scale with actual invocations and duration — genuinely cheaper at low/spiky volume, but can become *more* expensive than reserved compute at very high, sustained, predictable volume. Knowing this trade-off and saying it unprompted is a strong senior signal.

12.2 Networking & Edge

Service	Problem it solves	When to use	When NOT to use	Alternatives
ALB (Application Load Balancer)	L7 load balancing — HTTP/HTTPS, path/host-based routing	Standard web/API traffic, need content-based routing, WebSocket support	Raw TCP/UDP, extreme low-latency non-HTTP needs	NLB
NLB (Network Load Balancer)	L4 load balancing — TCP/UDP, extremely high throughput, static IP	Non-HTTP protocols, ultra-low latency, need a fixed IP for allowlisting, millions of connections	You need content-based (path/header) routing	ALB
CloudFront	AWS's CDN	Static/dynamic content delivery close to users, DDoS absorption at the edge, works natively with S3/ALB origins	Content that's genuinely unique per-request with no cacheable component	Other CDNs (Cloudflare, Akamai, Fastly)
Route 53	Managed DNS, plus health-check-based routing (failover, geo, latency-based routing policies)	Multi-region failover, geo-routing users to the nearest healthy region, domain management	— (usually a default choice on AWS)	Other DNS providers
API Gateway	Managed API front door	REST/HTTP/WebSocket APIs, especially fronting Lambda, need built-in throttling/auth/usage plans without running your own gateway	Very high sustained throughput internal service-to-service traffic where a self-hosted gateway (Envoy/Kong) or direct service mesh is more cost-effective at scale	Self-hosted Kong/Envoy, ALB directly for simpler cases

12.3 Storage & Databases

Service	Problem it solves	When to use	When NOT to use	Alternatives
S3	Durable, scalable object storage	Any blob/file storage — images, videos, backups, data lake, static site hosting; 11 nines of durability	You need low-latency random-access reads/writes to structured data (it's not a database)	GCS/Azure Blob on other clouds
RDS	Managed relational databases (MySQL/ PostgreSQL/ etc.)	Standard relational workloads, want automated backups/ patching/Multi-AZ failover without managing it yourself	Extreme scale beyond what vertical scaling + read replicas can handle, or need Aurora-specific performance	Aurora, self-managed RDBMS on EC2
Aurora	AWS's cloud-native relational engine, MySQL/ PostgreSQL-compatible	Need higher throughput/availability than standard RDS, want storage that auto-scales and heals, read replicas with very low replication lag	Cost-sensitive workloads where standard RDS is genuinely sufficient — Aurora carries a premium	RDS
DynamoDB	Fully managed, serverless-scaling key-value/document store	Need predictable single-digit-ms latency at any scale, access patterns are known upfront (fetch by key), want zero database operations overhead	Complex relational queries, ad-hoc queries/joins across entities, evolving/ unknown access patterns	MongoDB (self-managed or Atlas), Cassandra
ElastiCache	Managed Redis or Memcached	Caching layer, session store, rate limiting, leaderboards (Redis sorted sets) — everything from Chapter 4's caching section, without operating Redis yourself	—	Self-managed Redis on EC2/EKS, MemoryDB (Redis-compatible, more durability-focused)

12.4 Messaging & Streaming

Service	Problem it solves	When to use	When NOT to use	Alternatives
MSK (Managed Streaming for Kafka)	Fully managed Kafka	Already committed to Kafka's semantics (ordering, replay, consumer groups) but don't want to operate brokers/ZooKeeper/KRaft yourself	Simpler point-to-point queueing needs where SQS is sufficient and cheaper operationally	Self-hosted Kafka on EKS (what you may run today), Kinesis
SQS	Managed point-to-point queue	Work distribution across a worker pool, decoupling producers/consumers, simple and cheap	Need replay, ordered multi-consumer-group streaming (Kafka/Kinesis territory)	Kafka/MSK, RabbitMQ
SNS	Managed pub/sub, fan-out	Broadcasting one event to multiple independent subscribers (often paired with SQS per subscriber)	Need message replay/retention beyond delivery	Kafka/MSK for replay-needing fan-out

12.5 Security & Identity

Service	Problem it solves	When to use	Interview usage
IAM	Fine-grained access control to AWS resources, for both humans and services	Always — the foundation of least-privilege access; every service-to-service AWS interaction (e.g., an EKS pod reading from S3) should go through a scoped IAM role, not long-lived static credentials	Mention IAM roles for service accounts (IRSA on EKS) when discussing how a service authenticates to AWS resources — a strong, specific signal you've actually operated this
KMS	Managed encryption key creation/rotation/access control	Encrypting data at rest (RDS, S3, EBS) with centrally managed, auditable keys, separate from the data's own access controls	Mention when discussing encryption at rest for regulated data (Chapter 10)
WAF	Filters malicious HTTP traffic at the edge	Public-facing APIs/websites, especially anything payment-adjacent or at meaningful scale	Pair with CloudFront/ALB in your architecture diagram as an explicit security layer (Chapter 10.7)

12.6 Observability

Service	Problem it solves	When to use	Alternatives
CloudWatch	Native AWS metrics, logs, alarms, dashboards	Default, zero-setup observability for AWS-native services; CloudWatch Alarms for basic alerting	Prometheus + Grafana (what you likely run today) for richer, more flexible querying and multi-cloud/on-prem consistency; many teams run both — CloudWatch for AWS-service-level metrics, Prometheus/Grafana for application-level metrics

12.7 Putting It Together — A Reference Architecture

A typical mid-scale product on AWS, mapped end-to-end (this is the shape you should be able to draw from memory and adapt):

```
User → Route 53 (DNS/geo-routing)
      → CloudFront (CDN, static assets + edge caching)
      → WAF (filter malicious traffic)
      → ALB (L7 load balancing, TLS termination)
      → API Gateway or directly to services (auth, rate limiting)
      → EKS/ECS services (business logic, stateless, horizontally scaled)
        ↳ ElastiCache (Redis – caching, sessions, rate-limit counters)
        ↳ RDS/Aurora (primary transactional data) [+ read replicas]
        ↳ DynamoDB (high-scale key-value access patterns)
      → MSK/SQS/SNS (async processing, fan-out, decoupling)
      → S3 (media/blob storage, backups, data lake)
Cross-cutting: IAM (access control) · KMS (encryption) · CloudWatch + Prometheus/Grafana + OTel (observability)
Resilience: Multi-AZ everywhere by default · Multi-region for genuinely justified cases
```

Interview question: "Walk me through why you'd choose Aurora over standard RDS, or DynamoDB over both, for a given workload."

Ideal senior answer: "Standard RDS is my default — it's simpler, cheaper, and sufficient for the vast majority of relational workloads with Multi-AZ for HA. I'd move to Aurora specifically when I need either higher write throughput than RDS's underlying EBS-backed storage comfortably handles, or Aurora's faster replica lag and storage auto-scaling — essentially when RDS's ceiling is a real, evidenced constraint, not a hypothetical one. I'd reach for DynamoDB instead of either when my access patterns are simple and known upfront — fetch-by-key, not ad-hoc relational queries — and I specifically want to never think about database capacity planning again, accepting the trade-off of a more rigid data-access model in exchange for that operational simplicity."

Chapter 12 Interview Drill

1. When would you choose ECS over EKS, given both are container orchestrators on AWS?
 2. Why might Lambda be more expensive than EC2/EKS at high sustained throughput, despite being "serverless"?
 3. Walk through the difference between ALB and NLB, and when each is the right choice.
 4. Explain IAM roles for service accounts (IRSA) in one sentence, and why it's preferred over static credentials.
 5. Draw (verbally) the reference architecture above from memory, and justify each layer.
-

Next → [Chapter 13: Architecture Patterns](#) — the full pattern catalog: CQRS, Saga, Outbox, circuit breaker, consistent hashing, service mesh, and more, each with problem/solution/trade-offs.